

Learning the Subsystem of Local Planning for Autonomous Racing

Benjamin Evans¹, Herman A. Engelbrecht¹ and Hendrik W. Jordaan¹

Abstract—The problem of autonomous racing is to navigate through a race course as quickly as possible while not colliding with any obstacles. We approach the autonomous racing problem with the added constraint of not maintaining an updated obstacle map of the environment. Several current approaches to this problem use end-to-end learning systems where an agent replaces the entire navigation pipeline. This paper presents a hierarchical planning architecture that combines a high level planner and path following system with a reinforcement learning agent that learns that subsystem of obstacle avoidance.

The novel “modification planner” uses the path follower to track the global plan and the deep reinforcement learning agent to modify the references generated by the path follower to avoid obstacles. Importantly, our architecture does not require an updated obstacle map and only 10 laser range finders to avoid obstacles. The modification planner is evaluated in the context of F1/10th autonomous racing and compared to a end-to-end learning baseline, the Follow the Gap Method and an optimisation based planner. The results show that the modification planner can achieve faster average times compared to the baseline end-to-end planner and a 94% success rate which is similar to the baseline.

I. INTRODUCTION

Autonomous racing is the problem of generating speed and steering references for a non-holonomic vehicle, that move the vehicle through a race course as quickly as possible. Successful approaches to autonomous racing have implemented a hierarchical approach by splitting the problem into global planning, generating a high-level plan before the mission begins, and local planning, following the global plan while avoiding obstacles [1]. Current approaches to local planning for autonomous racing, generally assume that a map of the obstacles is available or can be built [1], [2]. It is difficult to simultaneously achieve the dual objective on obstacle avoidance without a map of the obstacles and following a global plan because information about the environment is high dimensional and difficult to plan with.

In this paper, we address the problem of local planning in autonomous racing without maintaining an explicit obstacle map. We present our novel approach which is a hybrid local planner that integrates a path follower and a deep reinforcement learning (RL) agent that modifies the steering reference generated by the path follower to accomplish the dual objective of tracking a reference path while also smoothly avoiding obstacles. We call our approach a reference modification planner since we learn the subsystem of obstacle avoidance by modifying the steering angle. We evaluate our novel approach in the context of an F1/10th autonomous racing

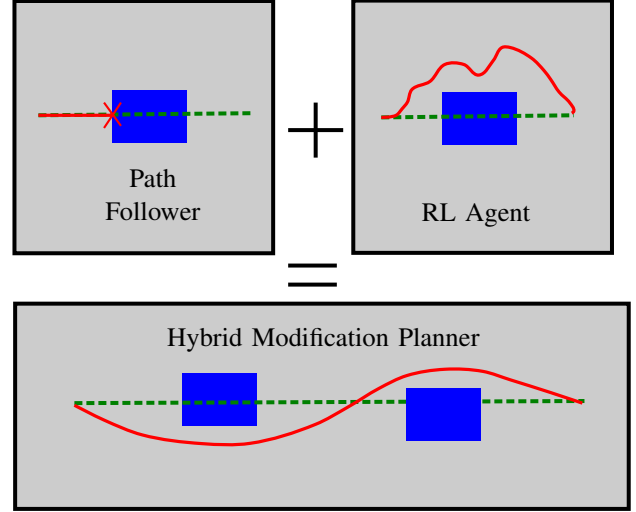


Fig. 1. **Illustration of Hybrid Planning Approach:** The green dotted line is the global plan, the blue blocks are the obstacles and the red line is the path taken by the vehicle. A path following algorithm that can follow a path is combined with an RL agent that learns the subsystem of obstacle avoidance to create the modification planner.

simulator and compare it to current approaches which include a end-to-end RL planner, the Follow the Gap Method, and an oracle optimisation based planner. The results show the advantages of our approach, namely, it does not require a map to avoid obstacles, and it can find smoother trajectories than end-to-end approaches.

The rest of the paper is summarised as follows. We first discuss previous attempts to solve the local planning problem for autonomous racing in Section II. This is followed by a theoretical overview of our proposed reference modification planner in Section III. In Section IV we explain the components that we use in building our solution and in Section V the learning formulation for the Reinforcement Learning (RL) agent is explained. Lastly in Sections VI and VII we detail the evaluation methodology and present our results.

II. RELATED WORK

Autonomous racing is built on the navigation problem of moving from a start to a goal location. We study different methods of local planning for navigation and racing to develop our solution. We study reactive methods [3], [4], classical planning techniques [5], [6], learning based approaches [7], [8] and hybrid systems [9], [10].

The earliest obstacle avoidance algorithms were reactive algorithms (such as “The Bug” algorithm) which generate steering references based on the surrounding environment at

¹Electrical and Electronic Engineering Department, Stellenbosch University, Stellenbosch, 7600, South Africa. {19811799, hebrecht, wjordan}@sun.ac.za;

each step [3]. One of the most recently proposed reactive obstacle avoidance algorithms is the “Follow the Gap Method” (FGM) [4]. The FGM method uses a dense LiDAR scan to navigate away from the nearest obstacle into free space. Reactive methods are not inherently able to follow a global plan and since they do not have a model of the vehicle, they are not able to plan for ahead.

Classical local planning methods aimed to improve on reactive methods by storing a map of the environment which can be used to plan a trajectory, multiple time-steps in advance. Classical methods typically build a map using a Simultaneous Localisation and Mapping (SLAM) algorithm [11], and then use the map to plan and optimise a collision-free trajectory [5]. The trajectory is then followed using a path tracking algorithm such as Pure Pursuit [6] or Model Predictive Control [2]. Classical planning and path tracking methods can generate and execute near-optimal trajectories, however, because map building is computationally expensive and slow, these solutions are not feasible for fast-moving, dynamic environments, such as autonomous racing, where all the processing must run on-board [12].

Due to the hardware requirements/constraints of classical planning solutions, machine learning (ML) methods have been applied to the problem of local planning for autonomous navigation [13]. Neural networks, trained with reinforcement learning (RL) [7] and imitation learning (IL) [8] have been used to associate sensor readings (camera images and LiDAR scans) directly with navigation commands. It has been shown that agents trained end-to-end are able to learn to race autonomously and perform competitively [14], [15]. While using an RL agent as a local planner does not require the expensive maintenance of a map, RL agents trained end-to-end can exhibit poor long term performance and have shown to require a high-level planner in certain circumstances [16].

Hybrid architectures that use RL agents to learn a subsystem of the planning pipeline have shown improved results over end-to-end methods [13]. Paths generated with A* search have been used in conjunction with a neural network as a local planner and shown to be able to execute long missions [10], [9]. Learning the subsystem of trajectory planning has shown to outperform end-to-end methods [17]. We further research into learning subsystems for autonomous racing by using an RL agent to learn the local planning subsystem of obstacle avoidance for autonomous racing. We aim to improve the performance of the vehicle by giving the agent less influence over the steering reference. Our solution doesn’t require an updated obstacle map, as with classical model-based planning solutions, and can harness the adaptability of learning techniques to form a planner which is able to smoothly avoid obstacles while maintaining a global trajectory.

III. THEORETICAL DESIGN

We address the problem of generating navigation references for autonomous racing that results in a reference trajectory being followed and obstacles being smoothly avoided as shown in Figure 1. The solution should track a reference

trajectory and deviate as necessary to safely and efficiently avoid the obstacle. The desired behaviour is analogous to driving a car, seeing an obstacle in the road, swerving around it and then returning to the original lane.

We present a hybrid local planner that features a pure pursuit path follower in parallel with an RL agent. Figure 2 shows how we combine the path follower and an RL agent in parallel such that the agent can modify the path follower references to avoid obstacles. This system is designed that the path follower maintains the reference path and the agent can deal with the uncertainty of obstacles.

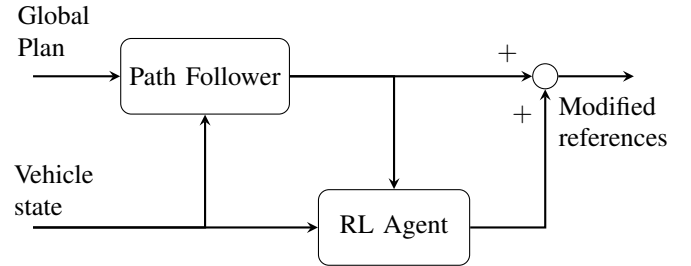


Fig. 2. **Proposed Local Planner Architecture:** A path follower to track the reference path is combined in parallel with an RL agent that modifies the path follower references.

The path follower uses the current location of the vehicle and a list of way-points to calculate the required steering and velocity commands to follow the global plan. An RL agent is added in parallel to avoid obstacles by modifying the steering angle calculated by the path follower. We train the agent by punishing it for crashing and rewarding it for progress along the global plan. These two components in the reward signal represent the dual objectives of achieving efficient, collision-free paths.

Where previous attempts have used RL agents to replace the local planner and learn to navigate, we use an RL agent to learn to navigate as a subsystem within our local planner. This is done with the aim of better utilising the global path and avoiding obstacles more smoothly. The aim is that the agent will learn to swerve around obstacles where needed without modifying the behaviour of the vehicle when no obstacles are present.

IV. PRELIMINARIES: NAVIGATION STACK

We design our system in the context of a standard perception, planning and control stack as shown in Figure 3. Before the race begins, the global planner uses a map of the environment to generate a reference path of way-points that connect the start and goal locations. The map does not contain any dynamic obstacles. During the race, the local planner and control system continuously operates to move the vehicle to the goal location.

We use a pure pursuit path follower which navigates towards a way-point situated at a fixed look-ahead distance in front of the vehicle. Figure 4 shows how the vehicle and way-points are modelled so that the steering angle can be calculated.

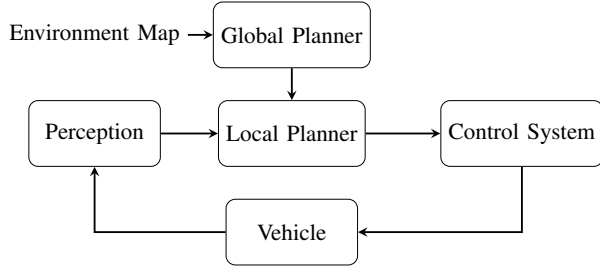


Fig. 3. Planning, Perception, and Control Navigation Stack

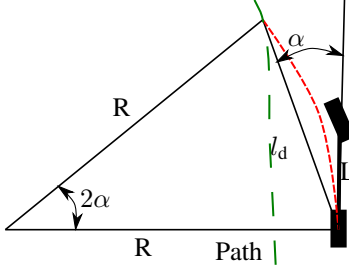


Fig. 4. **Pure Pursuit Path Follower:** α is the angle to the goal, l_d is the look ahead distance, L is the vehicle wheelbase, and R is the turning radius.

Using the notation from Figure 4, the desired steering angle, δ_{pp} is calculated according to,

$$\delta_{pp} = \arctan\left(\frac{2L \sin(\alpha)}{l_d}\right). \quad (1)$$

The velocity is calculated as the fastest the vehicle can safely travel while turning at the current steering angle. The velocity is calculated according to the maximum amount of lateral friction the vehicle can withstand, relative to the force exerted by the car in turning based on the current steering angle. We compute the velocity as,

$$V = f_s \sqrt{\frac{bmg}{m \tan(|\delta_{ref}|)/L}}, \quad (2)$$

where b is the coefficient of friction, m is the mass of the vehicle, L is the wheelbase length, and f_s is a safety factor. The velocity is limited to the vehicle's maximum velocity.

The navigation stack uses a low-level proportional control system to command the onboard vehicle actuators to follow the local planning references. The control system calculates acceleration and steering-velocity commands which can be implemented on hardware.

V. LEARNING POLICIES FOR REFERENCE MODIFICATION

The RL agent comprises a neural network that is trained from experience to maximise a reward signal [18]. The neural network is used to represent a policy that maps states to actions. For each training step, the network receives a state and selects an action that is implemented in the environment as shown in Figure 5. The environment then returns the updated state and a reward to the agent and the process is repeated.

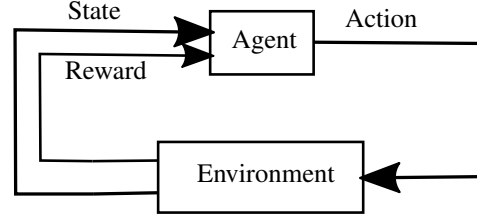


Fig. 5. Reinforcement Learning Agent Interacting With Environment

We train the agent to modify the pure pursuit steering reference δ_{pp} to prevent collisions. The state which the network receives consists of a vector containing the current velocity V_t and steering of the vehicle δ_t , the references calculated by the path follower δ_{pp} , and the current range finder scan $[r_1, \dots, r_{10}]$. The network outputs an action quantity which is used to modify the steering reference calculated by the path follower. The path follower (pp) and neural network (nn) steering commands are combined as,

$$\delta_{ref} = \delta_{pp} + \delta_{nn}. \quad (3)$$

The reference steering value is limited to be within the maximum range and then sent to the control system to be executed. The inputs and outputs from the network are in the range $[-1, 1]$ and are scaled according to the maximum values.

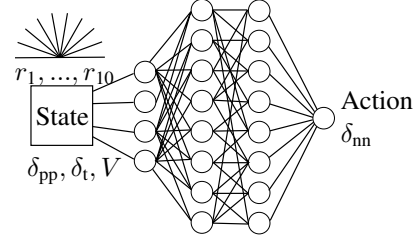


Fig. 6. **Reinforcement Learning Agent Architecture:** The state consists of 10 sparse laser range finders and the pure pursuit steering reference. A neural network with two fully connected layers is used to calculate an action which is the steering reference which is used to modify the path follower reference.

The aim in training the network is that the vehicle does not crash and that it tracks the reference path closely. Therefore, we use a reward signal which severely punishes crashing and rewards progress along the reference path. We write our reward signal as,

$$r(s_t, a_t) = \begin{cases} r_{crash} = -1 & \text{if crash} \\ r_{complete} = 1 & \text{if lap complete} \\ r_{tracking} = (s_{t+1} - s_t)/s_{total} & \text{otherwise,} \end{cases} \quad (4)$$

where s is the progress along the track at a specific step. The sum of the tracking rewards for completing the episode will always be 1 since the sum of the progress differences will be equal to the track length. The tracking reward must be a potential function of the terminal reward so that the agent is rewarded in proportion to the progress towards the ideal terminal state.

VI. EVALUATION METHODOLOGY

The ability of the modification planner to follow a reference path and avoid obstacles efficiently is evaluated in the context of F1/10th autonomous racing. We aim to determine the performance of the modification planner and compare it to current baseline solutions. We measure performance in terms of the following metrics, i.e. (1) the time to navigate to a target and (2) the success rate, which is the percentage of episodes successfully completed without crashing.

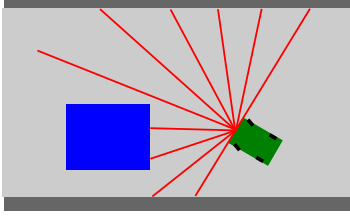


Fig. 7. F1/10th Vehicle Simulator with 10 Laser Range Finder Beams

A. Simulation Environment

We train and test our local planner architectures using a custom-built, F1/10th autonomous racing simulator. The simulator models a non-holonomic 1/10th scale autonomous race car with a LiDAR sensor as shown in Figure 7. At each time-step, the simulator receives an action, updates the state according to the transition dynamics and then returns the new state of the vehicle. A version of our test code is available online ¹.

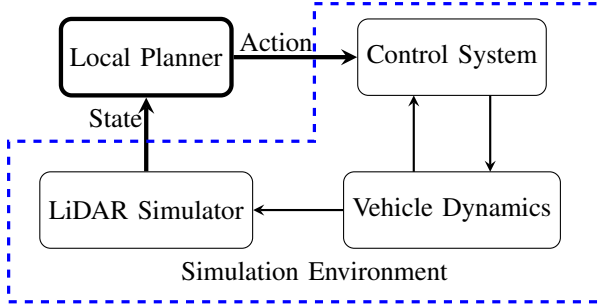


Fig. 8. Schematic of Planning Loop in Simulation Environment

The car is modelled using the kinematic bicycle model [19], with the vehicle state consisting of, position, orientation, velocity, and steering angle. Figure 8 shows how the simulator interfaces with the local planner that is being evaluated. The local planner selects an action consisting of velocity and steering angle commands which are passed to the simulator and used as the references for the control system. The feedback control system calculates acceleration and steering-velocity commands which are the input to the dynamics equations. As in practice, the simulated control system runs at a higher frequency of 100Hz compared to the local planner which runs at 10Hz. Table I notes several important parameters used in our simulation.

Simulation Parameter	Value
Simulation dynamics time step	0.01 s
Local planning time step	0.1 s
Max range finder value	10 m
Range finder noise σ_s	0.01
Max vehicle speed	7 m/s
Max steering angle	0.4 rad
Pure Pursuit Look-ahead distance	0.8 m

TABLE I
PARAMETERS USED IN THE SIMULATION

The new vehicle state is used to simulate a LiDAR scan of 10 sparse range finders on the map and the new state and the scan is returned to the local planner. To simulate the noise in range finder readings, Gaussian distributed process noise, $\mathcal{N}(0, \sigma_s)$, is added to the measurements.

B. Baseline Solutions

We compare the modification planner to three common baseline local planners. The first baseline is the “Follow the Gap Method” (FGM) [4]. The FGM method identifies a bubble around the nearest obstacle and then navigates away from the bubble into free space. In order for the FGM method to accurately identify a gap, it requires a dense LiDAR scan of many beams. We thus adapt the simulator to produce 1080 beams.

The second baseline which is used is an optimisation based, oracle planner which has full access to a map of the environment and the location of the obstacles before the episode begins. The obstacle map is used to generate an optimal path around the obstacles. The optimal plan is followed using the pure pursuit algorithm. This solution is treated as the best performance which could be achieved by any local planner.

The main baseline that we compare our solution to is a end-to-end planner that uses an RL agent to replace the entire navigation pipeline. The end-to-end planner uses the relative target location and laser range finder readings to calculate a steering angle that leads the vehicle around obstacles towards the target. The end-to-end planner is trained in the same way as the modification planner using the same neural network parameters and reward signal.

For all of our baseline implementations, we use the same method of calculating the speed based on the steering angle to ensure that the results are comparable.

C. Reinforcement Learning

The RL agent in the modification planner is trained using the Twin Delayed Deterministic Policy Gradient (TD3) reinforcement learning algorithm with the original training hyper-parameters [20]. The TD3 algorithm is selected since it is a current state of the art algorithm in many RL challenges. Neural networks with two hidden layers of 200 neurons are trained in mini-batches of 100 transitions. The training lasts for 200,000 steps which is generally just below 4000 episodes. 200,000 steps was selected after evaluating the agent after different numbers of steps to ensure and seeing

¹Available at: <https://github.com/BDEvan5/ReferenceModification>

that it had always converged by 200,000. Figure 9 shows the average reward per episode for the training of the network. The network converges towards a value of 2 which is made up from the total of the tracking reward and terminal reward.

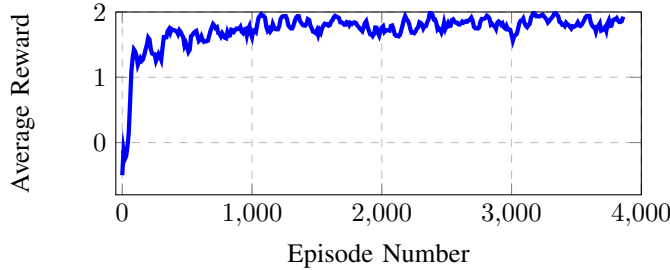


Fig. 9. Modification Planner Training Graph

VII. EVALUATION RESULTS

A. Random Forests

The modification planner is evaluated in a random forest which consists of a straight race track with randomly located obstacles. The random forest is a straight track (20m x 2m) where 4 obstacles (0.5m x 0.5m) are randomly placed every episode along the track. Table II shows the results from running 100 test episodes and taking the average times to navigate to goal and success rate.

Vehicle	No Obstacles	Obstacles	Success
Follow the Gap Method	3.73	4.38	99%
Oracle	3.72	3.87	100%
End-to-end Planner	5.08	6.18	97%
Modification Planner	3.82	4.63	94%

TABLE II
RESULTS FROM 20M RANDOM FOREST

In the tests with no obstacles the modification planner is able to track the optimal path, straight line through the forest, and achieve a very similar time to the optimisation based oracle planner (within 3% of the oracle planner). The similar times to the optimisation based planner when no obstacles are present shows that the RL agent learns to not modify the steering command unnecessarily. The end-to-end planner achieves a significantly slower average time of 5.08 seconds in this test compared to the 3.72 seconds which the other planners are able to achieve.

In the tests with obstacles present, the optimisation based oracle planner outperforms the other planners as expected with an average time of 3.87 seconds. The FGM planner achieves a mediocre time of 4.33 seconds, however, this is still faster than any of the learning-based planners. Reactive methods perform well in highly structured environments where the best route is straight forward, and complex velocity profiles are not required. The modification planner achieves an average time of 4.63 seconds for the forest filled with obstacles which is faster than the end-to-end planners time

of 6.18 seconds. The decrease in time shows that the modification planner is able to find smoother, faster trajectories than the end-to-end solution. Figure 10 shows a comparison of the trajectories taken by the end-to-end and modification planners and how the modification planner learns to take smoother trajectories.

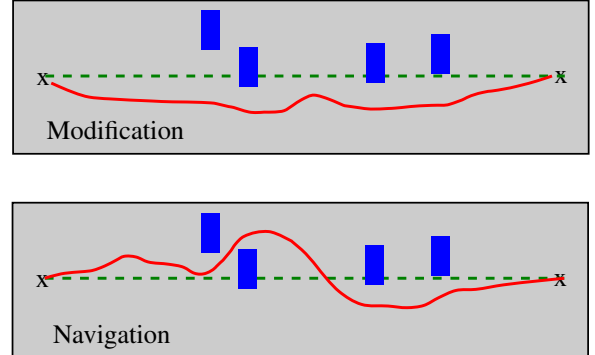


Fig. 10. Example Trajectories Executed by the Modification and End-to-end Planners

The modification planner achieves a slightly lower success rate of 94% compared to the end-to-end planner's 97%. These high success rates show that both planners learn to avoid a significant majority of obstacles. Many similar papers in the field of autonomous racing show the similar result of learning to race, yet still occasionally crashing [14], [15].

The influence of the network in modifying the path follower references is shown below. Figure 11 shows a graph of the output from the neural network as the vehicle moves along the path. The neural network modifies the path enough to avoid the obstacles while roughly maintaining the reference path.

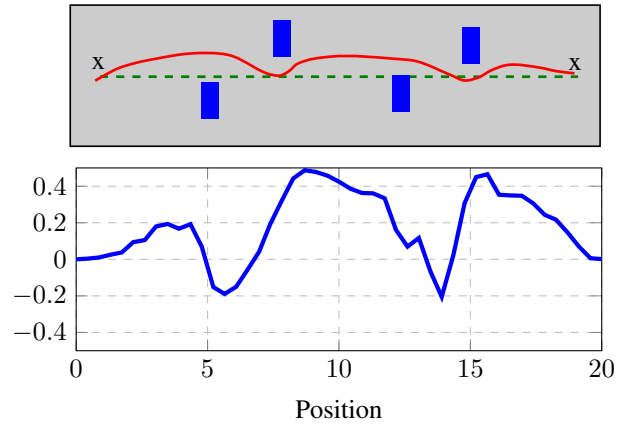


Fig. 11. Example Trajectory from the Modification Planner with Neural Network Output. The y-axis label (hidden due to size) is the scaled output from the neural network.

B. Repeatability

To measure the repeatability of the learned policies, we train and evaluate 10 agents on the random forest described above. The average and standard deviation statistics are presented in Table III.

Statistic	Average	Standard Deviation
No Obstacle Average Time	3.85 s	± 0.10
Obstacle Average Time	4.63 s	± 0.17
Obstacle Success Rate	92.2%	± 5.3

TABLE III

RESULTS FROM REPEATABILITY EVALUATION OF THE MODIFICATION PLANNER

The standard deviation in the times achieved by the different agents is small and this shows that the method of training agents to achieve similar times is stable and highly repeatable. The standard deviation of the success rate is large, 5%, which shows that there is a non-trivial difference in how agents learn to avoid obstacles.

C. Discussion

The results show that the modification planner can learn a policy that produces faster performance than the end-to-end planner. Specifically, the modification planner avoids obstacles more smoothly which results in the vehicle being able to travel at higher speeds. This shows that the modification planner achieves the behaviour that it was designed for, allowing non-holonomic vehicles to avoid obstacles smoothly at high speeds. The success rate which the planners achieve show that while the modification planner is able to learn to avoid a significant number of obstacles, it still underperforms the end-to-end planners ability to consistently avoid obstacles.

It is suggested that the reason for the lower rate of completion is that the complexity of the learning problem for the modification planner is higher than for the end-to-end baseline. The transition dynamics (movement from one state to the next) for the modification planner are dependant on combining the action, which the agent has selected, with the pure pursuit reference. The result of this combination is that the agent has less influence on how the vehicle moves. Having less influence improves the quality of the path taken which results in the modification planner achieving faster times. A problem that arises in having less influence is that it achieves a lower rate of completion.

This problem of guaranteeing vehicle safety is a current topic of research and could be solved in future work by exploring different training methods such as imitation learning. Another possible solution is to combine the RL agent with a safety system that can stop the vehicle before it crashes.

A recent survey [13] suggested that future work in mobile navigation should continue into using ML to learn specific subsystems of the navigation pipeline. In future, we plan to investigate how RL agents can be used in different architectures to learn obstacle avoidance behaviours by reducing the complexity of the navigation problem to more simple components.

VIII. CONCLUSION

In this paper, we addressed the problem of autonomous racing without an explicit obstacle map. We presented a

novel planning architecture that uses a path following algorithm in conjunction with an RL agent to smoothly avoid obstacles while maintaining the global plan. Our architecture is evaluated in the context of F1/10th autonomous racing and shown to achieve comparable racing times to optimisation based and reactive methods, yet using fewer inputs. Specifically, our modification local planner requires only 10 sparse laser range finder readings and is able to smoothly avoid obstacles. We showed that our planner is able to avoid obstacles more smoothly than previous solutions that were trained end-to-end.

REFERENCES

- [1] J. Betz, A. Wischniewski, A. Heilmeier, F. Nobis, L. Hermansdorfer, T. Stahl, T. Herrmann, and M. Lienkamp, "A software architecture for the dynamic path planning of an autonomous racecar at the limits of handling," in *2019 IEEE International Conference on Connected Vehicles and Expo (ICCVE)*. IEEE, 2019, pp. 1–8.
- [2] J. L. Vázquez, M. Brühlmeier, A. Liniger, A. Rupenyan, and J. Lygeros, "Optimization-based hierarchical motion planning for autonomous racing," *arXiv preprint arXiv:2003.04882*, 2020.
- [3] V. J. Lumelsky and A. A. Stepanov, "Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape," *Algorithmica*, vol. 2, no. 1-4, pp. 403–430, 1987.
- [4] V. Sezer and M. Gokasan, "A novel obstacle avoidance algorithm: 'follow the gap method'," *Robotics and Autonomous Systems*, vol. 60, no. 9, pp. 1123–1134, 2012.
- [5] L. Heng, L. Meier, P. Tanskanen, F. Fraundorfer, and M. Pollefeys, "Autonomous obstacle avoidance and maneuvering on a vision-guided mav using on-board processing," in *2011 IEEE International Conference on Robotics and Automation*. IEEE, 2011, pp. 2472–2477.
- [6] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie-Mellon UNIV Pittsburgh PA Robotics INST, Tech. Rep., 1992.
- [7] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 31–36.
- [8] G. Kahn, T. Zhang, S. Levine, and P. Abbeel, "Plato: Policy learning using adaptive trajectory optimization," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 3342–3349.
- [9] W. Gao, D. Hsu, W. S. Lee, S. Shen, and K. Subramanian, "Intention-net: Integrating planning and deep learning for goal-directed autonomous navigation," in *Conference on Robot Learning*. PMLR, 2017, pp. 185–194.
- [10] M. Wei, S. Wang, J. Zheng, and D. Chen, "Ugv navigation optimization aided by reinforcement learning-based path tracking," *IEEE Access*, vol. 6, pp. 57 814–57 825, 2018.
- [11] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: part i," *IEEE robotics & automation magazine*, vol. 13, no. 2, pp. 99–110, 2006.
- [12] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.
- [13] X. Xiao, B. Liu, G. Warnell, and P. Stone, "Motion control for mobile robot navigation using machine learning: a survey," *arXiv preprint arXiv:2011.13112*, 2020.
- [14] E. Perot, M. Jaritz, M. Toromanoff, and R. De Charette, "End-to-end driving in a realistic racing game with deep reinforcement learning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2017, pp. 3–4.
- [15] M. Vitelli and A. Nayebi, "Carma: A deep reinforcement learning approach to autonomous driving," *Tech. rep. Stanford University*, 2016.
- [16] F. de Villiers and W. Brink, "Learning fine-grained control for mapless navigation," in *2020 International SAUPEC/RobMech/PRASA Conference*. IEEE, 2020, pp. 1–6.
- [17] T. Weiss and M. Behl, "Deeppracing: Parameterized trajectories for autonomous racing," *arXiv preprint arXiv:2005.05178*, 2020.

- [18] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [19] P. Polack, F. Altché, B. d’Andréa Novel, and A. de La Fortelle, “The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles?” in *2017 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2017, pp. 812–818.
- [20] S. Fujimoto, H. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International Conference on Machine Learning*. PMLR, 2018, pp. 1587–1596.